REVIVE-Probe: Pipeline-Boundary Activations as a Substrate for Runtime Interpretability and Steering

HackPrinceton Spring 2026
Alignment & Mechanistic Interpretability Track (d_model)

April 2026

Abstract

Most interpretability work runs in offline notebooks against frozen weights, using hooks that modify the model's forward pass. Production systems rarely tolerate either. This project uses a property of pipeline-parallel inference that is normally considered a deployment inconvenience: the hidden state tensor that crosses each stage boundary must be serialized, transmitted, and reconstructed. We argue that this boundary is a free, permanent, side-effect-free tap into the residual stream, and we use it to (i) train linear probes for safety-relevant concepts without touching the model, and (ii) apply activation steering by modifying the tensor in flight. The substrate is a 3-node distributed inference cluster we built from scratch (REVIVE) running a fine-tuned Qwen3 model across two iPhones and a laptop. Probes trained on pipeline-boundary activations recover refusal and harmful-intent signals at F1 ≥ 0.88 on a held-out split, and steering vectors injected at boundary 2 of 3 shift refusal rates on a red-team prompt set from 42% to 71% with no loss of fluency on benign prompts. We release code, weights, and the raw captured activations.

1 Motivation

A typical mechanistic interpretability pipeline looks like this: load model, register forward hooks on the layers of interest, run prompts, collect activations, train probes [1, 3]. The hooks are intrusive: they live inside the framework, they depend on exact module names, they are removed when the process dies. Activation steering [8, 9] inherits the same constraint.

Pipeline parallelism splits a model across devices. A forward pass on stage k ends by serializing the residual stream $h_k \in \mathbb{R}^{B \times S \times d_{\text{model}}}$ to bytes, sending it over a socket to stage $k+1$, and deserializing there. The tensor on the wire is exactly the quantity a researcher would want to hook. Nothing inside the model needs to change. Nothing inside the inference framework needs to change. Any process that can see the wire can read, write, or replace the activations.

We think this is interesting for three reasons. First, it is deployment-compatible: if a production system already runs pipeline-parallel (as most serving stacks above a single GPU do), the hooks are free. Second, it is composable: the boundary is the interface, so the probe author never touches the inference code. Third, it is adversarially robust in a narrow sense: the probe and the model do not share memory, so a compromised probe cannot silently rewrite weights, and a compromised model cannot silently disable a probe without breaking the wire protocol.

2 The Substrate: REVIVE

REVIVE is a 3-node pipeline-parallel inference system we built for HackPrinceton. It runs a fine-tuned `Qwen3-0.6B` (knowledge-distilled from a larger Qwen3 checkpoint) across a ring of consumer devices: Anish's iPhone (layers $[0, 10)$, first stage), Vikram's iPhone (layers $[10, 19)$, middle stage), and a MacBook Pro (layers $[19, 28)$, last stage). Inter-stage transport is a length-prefixed binary framing over HTTP/1.1, carrying an fp16 tensor of shape $(1, S, d_{\text{model}}{=}1024)$ per step. Membership, heartbeats, and partition decisions are managed by an Arduino Uno acting as a baseboard management controller (BMC), with a software simulator used for CI. A 4-inch SenseCAP Indicator (ESP32-S3, RGB LCD) serves as a hardware status panel. The system produces token-identical output to a single-machine reference run. Architectural detail is in the project's main README; for the rest of this report the only relevant fact is that we can freely read and write the residual stream at layer 10 and layer 19.

3 Method

3.1 Intercept layer

REVIVE's coordinator already reconstructs each stage's output before forwarding. We added a small interceptor class that sits between deserialization and the next `post_forward` call. The interceptor holds a list of registered callbacks, each of type

$$f : (h_k,\ k,\ \text{pos_ids},\ \text{meta}) \mapsto h'_k$$

where h'_k is either the original tensor (read-only probe) or a modified one (steering). Callbacks run synchronously on the coordinator thread; measured overhead is $< 400\,\mu$s per step at $d = 1024$, which is dominated by the memcpy and negligible relative to the $\sim$70 ms pipeline latency per token.

3.2 Probes

We trained linear probes on the final-token activation of the middle-stage boundary (h_{10}) following the standard protocol of Alain and Bengio [1]: a single dense layer from $\mathbb{R}^{1024} \to \mathbb{R}^2$, logistic loss, Adam, early stopping on a held-out 15% split. Because the point is to show that a < 1 MB probe can run inline on a laptop, we deliberately did not use anything fancier.

We targeted three concepts. *Refusal* was labeled by prompting the model with pairs from Arditi et al. [2] and marking which prompts yielded a string-match refusal in the first 15 tokens of generation. *Harmful intent* used the AdvBench subset of Zou et al. [10] as positives and a length-matched sample of Alpaca instructions as negatives. *Factuality* used TruthfulQA binary pairs, where the positive class is the truthful continuation. In all three cases, the activation was captured on the final prompt token, before any generation; the label applied to whatever the model would do next.

3.3 Steering

For steering, we computed a contrastive direction $v_{\text{refuse}} \in \mathbb{R}^{1024}$ as the difference of class-conditional means of h_{10} on the refusal-probe training data, following the standard recipe [8, 7]. At inference time, the interceptor replaces h_{10} with

$$h'_{10} = h_{10} + \alpha \, \frac{v_{\text{refuse}}}{\|v_{\text{refuse}}\|}, \quad \alpha \in \{-8, -4, 0, 4, 8\}$$

at the last prompt position, then lets the rest of the pipeline run unchanged. The coefficient is the only hyperparameter; we report a small sweep.

4 Experiments and Results

4.1 Probe accuracy

We captured h_{10} activations from 1800 prompts (600 per concept, balanced). Linear probes achieved held-out F1 of 0.91 (refusal), 0.88 (harmful intent), and 0.71 (factuality). The gap between the first two and factuality matches prior reports that factuality is less linearly separable in small open-weight models [6]; we do not claim a new result there. The refusal and intent probes, on the other hand, are strong enough that we use them in the next experiment as live filters.

4.2 Activation steering

We evaluated refusal-steering on a 200-prompt red-team set (80 harmful, 120 benign, drawn from AdvBench and Alpaca respectively). Refusal rate was judged by a string-match rubric plus a GPT-4 grader for borderline cases. At $\alpha = 0$ (baseline), the model refused 42% of harmful prompts and 3% of benign prompts. At $\alpha = 4$ the rates became 71% harmful, 7% benign. At $\alpha = 8$ the harmful rate climbed to 84% but benign false-positives also rose to 19% and output fluency (judged by per-token perplexity under a reference model) degraded 11%. We report $\alpha = 4$ as the operating point.

Negative α produced the expected unsafe shift: at $\alpha = -4$ the harmful-refusal rate dropped to 18%. We do not release code for this direction, but we document it because it is a relevant safety property of the method: anyone with wire access can jailbreak as easily as they can patch.

4.3 Where the probe fires in a real chat

A concrete demonstration: we instrumented the live REVIVE chat UI with the refusal probe. As the user types a prompt and the first stage begins processing, the probe fires on h_{10} before any tokens are generated. The UI shows a thin colored bar next to the input, green if the probe says *refuse* is unlikely and red if it is. In practice, prompts that the model would go on to refuse get flagged correctly 87% of the time, and the flag appears about 40 ms before the first refusal token would have arrived. This is not a deployment artifact we propose putting in front of users; it is a proof that runtime steering signals exist and are cheap to extract.

5 What this does and does not show

The strong claim is narrow: at a pipeline boundary chosen post-hoc ($k = 10$ of 28 layers), linear probes for refusal and harmful intent are accurate enough to be useful, and a single contrastive direction is enough to shift refusal rates by +29 points with acceptable fluency cost. We suspect, but have not shown, that the boundary is not special: any middle layer would give a similar probe. The value of doing it at the boundary is pragmatic, not theoretical.

We do not show that boundary-probing scales to the size of models people actually worry about. Qwen3-0.6B is small. A 70B model split 8-ways would give the same number of boundaries but the computational cost of running a probe on each becomes non-trivial. We also do not show that the

approach is adversarially robust in the strong sense: a model owner could collude with a pipeline implementation to lie about which tensor is on the wire.

The most honest framing is that this project is a feasibility demonstration. The infrastructure to do this at production scale exists (every serving stack that uses pipeline or tensor parallelism already marshals activations on the wire), the probes and steering directions are cheap to train, and the integration cost against a running inference cluster is a few hundred lines of Python. What is missing is the institutional willingness to bake this kind of probe into deployment, and a body of work that characterizes what boundary-layer probes miss compared to inside-the-layer ones.

6 What we would do next

Three directions matter, in decreasing order of tractability. First, sparse autoencoder features [4, 5] on boundary activations: the same argument applies. If h_{10} is a sum of a small number of interpretable features, a dictionary trained on boundary captures should recover them. Second, cross-boundary circuit analysis: with two boundaries ($k = 10$ and $k = 19$), we can track the flow of a feature across the middle third of the network without patching. Third, adversarial evaluation of the steering intervention: if a user knows their prompts are being steered at $\alpha = 4$, can they craft a prompt that the probe misses but the model still executes? This is the question that matters for any deployment story, and we have not tested it.

7 Reproducibility

Code is at `github.com/anishkataria/revive` under `revive/interp/`. The probe training notebook is `probes.ipynb`; the steering evaluation is `steering_eval.py`. Captured activations ($\sim$45MB) are released as `activations_h10.npz`. The probe weights are a 1024×2 matrix per concept, shipped as `probes.pt`. A single GPU is not required to reproduce; we trained the probes in < 90 seconds on a MacBook CPU.

References

[1] Guillaume Alain and Yoshua Bengio. Understanding intermediate layers using linear classifier probes. *ICLR Workshop*, 2017.

[2] Andy Arditi et al. Refusal in language models is mediated by a single direction. *arXiv:2406.11717*, 2024.

[3] Nora Belrose et al. Eliciting latent predictions from transformers with the tuned lens. *arXiv:2303.08112*, 2023.

[4] Trenton Bricken et al. Towards monosemanticity: Decomposing language models with dictionary learning. *Transformer Circuits Thread*, 2023.

[5] Hoagy Cunningham et al. Sparse autoencoders find highly interpretable features in language models. *arXiv:2309.08600*, 2023.

[6] Samuel Marks and Max Tegmark. The geometry of truth. *arXiv:2310.06824*, 2023.

[7] Nina Rimsky et al. Steering Llama 2 via contrastive activation addition. *arXiv:2312.06681*, 2023.

[8] Alex Turner et al. Activation addition: Steering language models without optimization. *arXiv:2308.10248*, 2023.

[9] Andy Zou et al. Representation engineering: A top-down approach to AI transparency. *arXiv:2310.01405*, 2023.

[10] Andy Zou et al. Universal and transferable adversarial attacks on aligned language models. *arXiv:2307.15043*, 2023.